

Machine Learning Paradigms Mapped to Real-World Problems

A Staff+ Engineer's Interview & Architecture Handbook

Audience · Senior / Principal / Staff engineers
Use · ML systems design, AI platform interviews
Reads in · 60–90 minutes
Version · 2026.05

1. EXECUTIVE SUMMARY — 21 PARADIGMS AT A GLANCE

PARADIGM	EXAMPLE PROBLEMS	TYPICAL MODELS	INFRA PATTERN	MODERN RELEVANCE
Supervised	Fraud, churn, credit risk, spam, CTR, lead scoring	LogReg, GBDT (XGBoost / LightGBM), CNN, BERT-finetune	Batch train → offline + online infer; feature store	~70% of enterprise ML
Unsupervised	Segmentation, log clustering, dim reduction, EDA	K-Means, DBSCAN, GMM, PCA, UMAP	Batch / notebook; pre-step in pipelines	Rarely standalone in prod
Semi-supervised	Rare-label medical imaging, fraud rings	Self-training, MixMatch, FixMatch	Pseudo-labels feed retrain loop	Niche; bridged by self-supervised
Self-supervised	LLM / vision foundation-model pretraining	BERT, GPT, MAE, CLIP, SimCLR, DINO	Massive GPU clusters, sharded data lake	Foundation of all GenAI
Reinforcement Learning	Robotics, ads bidding, RLHF, sys tuning	DQN, PPO, A3C, MuZero, AlphaZero	Simulator + replay buffer; long training	RLHF / RLAI for LLMs; rare end-to-end
Deep Learning	Vision, speech, language, recsys	CNN, RNN, Transformer, MLP, DLRM	GPU / TPU, distributed, mixed precision	Unified by Transformers
Transfer Learning	Domain classifiers with small labels	Fine-tune BERT / CLIP / ResNet; LoRA, QLoRA	Pretrain → fine-tune; PEFT; checkpoint registry	Cheap path to domain models
Online Learning	Ads CTR, real-time pricing, news ranking	FTRL, Vowpal Wabbit, online GBDT	Streaming features (Kafka / Flink); warm models	Critical in high-velocity domains
Active Learning	Medical annotation, doc review, eval-set curation	Uncertainty / margin / committee sampling	Human-in-the-loop labeling service	Core of data-centric ML
Federated Learning	Mobile keyboards, on-device health, banks	FedAvg, FedProx, Secure Aggregation	Edge training + secure aggregation	Privacy niches (Gboard, iOS)
Recommendation	E-com, video, music, social feeds	MF, NCF, Two-Tower, SASRec, DLRM	Candidate-gen → ranking; offline + online	Hidden core of consumer AI revenue
Ranking (LTR)	Search, ads, recsys final stage	LambdaMART, XGBoost-Rank, DLRM, BERT-rerank	Retrieve → rank → re-rank, p99<100 ms	Where most \$ is decided
Time-Series Forecasting	Demand, capacity, finance, SRE telemetry	ARIMA, Prophet, GBDT, TFT, N-BEATS, DeepAR	Batch retrain + rolling backtest	Boosting + DL hybrids dominate
Anomaly Detection	Fraud, intrusion, ops alerts, drift	Isolation Forest, AE, LOF, OC-SVM, robust z-score	Stream (Kafka → Flink) + batch baselines	Core in security & observability
NLP (classical)	Classification, NER, summarization, QA	BERT, RoBERTa, T5, DistilBERT, fastText	Pretrained + fine-tune / prompt	Classifiers still cheaper than LLMs
Computer Vision	OCR, detection, medical imaging, AV	ResNet, EfficientNet, ViT, YOLO, SAM, DINO	GPU inference; edge runtimes (TensorRT)	Hybrid CNN + ViT + foundation
Graph ML	Fraud rings, knowledge graphs, drug discovery	GCN, GAT, GraphSAGE, PinSAGE	Distributed graph store + GPU sampling	Niche but powerful relational
Embeddings & Vector Search	Semantic search, dedup, retrieval, recsys	SBERT, E5, BGE, OpenAI / Cohere, multimodal	ANN index (HNSW, IVF-PQ); vector DB	Foundation of RAG & modern search
Generative AI	Content gen, copilots, summarization, code	LLMs (GPT, Claude, Llama), diffusion (SD, Flux)	Inference servers (vLLM, TGI), GPU pools	Dominant for unstructured tasks
RAG Systems	Enterprise Q&A, copilots, grounded chat	Embedder + retriever + reranker + LLM	Chunk → embed → ANN → rerank → LLM	Default enterprise GenAI pattern
Multi-Agent AI	Workflow automation, research, ops bots	LLM + tools + planner / critic	Orchestrator (LangGraph, CrewAI) + eval	Emerging; brittle, hard to evaluate

How to read this: Sections 2.A–G repeat per paradigm but kept lean for the obvious ones. The most production-relevant paradigms — recsys, ranking, embeddings, RAG, anomaly detection, forecasting, online inference, MLOps, feature stores — get larger sections starting on p. 4.

2. FOUNDATIONAL PARADIGMS (COMPACT)

Supervised Learning

WORKHORSE

Problem: learn $f(x) \rightarrow y$ from labeled examples. Classification + regression.

Production: fraud (Stripe / Visa), churn, credit risk, ad CTR, spam, ETA prediction (Uber), lead scoring.

Models: LogReg (baseline + interpretable), **GBDT** (XGBoost / LightGBM) — dominant on tabular; CNN / BERT only when unstructured.

Infra: batch train (Airflow / Kubeflow); offline + online inference; feature store enforces train/serve parity; retrain weekly-daily.

Metrics: AUC, PR-AUC, log-loss, calibration; *not* accuracy on imbalanced data.

GenAI link: LLMs do *not* replace tabular GBDTs — slower, costlier, less calibrated.

Use when labels exist & features are structured.

Watch: label leakage, class imbalance, drift.

Unsupervised Learning

EXPLORATION

Problem: structure without labels — clusters, density, low-dim manifolds.

Production: customer cohorts, log / trace clustering (Datadog), document de-dup, image dedup at scale, embedding-space exploration.

Models: K-Means (cheap, spherical), DBSCAN (density, noise), GMM (soft clusters), PCA / UMAP / t-SNE (viz).

Infra: usually batch / notebook. As a *pre-step* in pipelines (segmentation feeding supervised models).

Metrics: silhouette, Davies-Bouldin; ultimately judged by downstream lift.

Trap: "clusters" without business meaning. Always validate against an action.

Semi-Supervised Learning

LABEL-SCARCE

Problem: small labeled set + large unlabeled set. Bridge to supervised.

Production: medical imaging (few annotated scans), document classification, fraud-ring discovery, content moderation bootstrap.

Models: self-training, pseudo-labeling, MixMatch / FixMatch, label propagation.

Infra: iterative — model predicts $\rightarrow$ high-confidence preds become labels $\rightarrow$ retrain. Active-learning loop common.

Modern reality: often replaced by *fine-tuning a self-supervised foundation model*.

Self-Supervised Learning

PRETRAINING

Problem: create supervision *from the data itself* (masked tokens, next-token, contrastive pairs).

Production: every modern foundation model — BERT (MLM), GPT (next-token), CLIP (image-text contrastive), MAE (masked image), DINO / SimCLR.

Infra: 100s–10000s of GPUs/TPUs, sharded data, FSDP / DeepSpeed / Megatron, weeks of training, multi-region.

Why it matters: turns the world's unlabeled data into reusable representations — the substrate of all GenAI.

Most teams never *do* SSL; they fine-tune the output (transfer learning).

Reinforcement Learning

SEQUENTIAL DECISIONS

Problem: agent learns policy to maximize cumulative reward through interaction.

Production: RTB ads bidding, robotics (Boston Dynamics, Tesla FSD), datacenter cooling (DeepMind), DB index tuning, network congestion control, **RLHF / RLAIF** for LLM alignment.

Models: DQN (discrete), PPO (general, stable), SAC (continuous), MuZero (model-based), bandits (cheap RL surrogate).

Infra: simulators or offline logs; massive replay buffers; reward shaping is most of the work.

When RL fails: sparse rewards, off-policy distribution shift, unsafe exploration, no simulator $\rightarrow$ use supervised + bandits instead.

Deep Learning (umbrella)

UNSTRUCTURED DATA

Problem: learn hierarchical features from raw signals (pixels, audio, text, sequences).

Models: CNN (local spatial), RNN / LSTM (sequence, mostly legacy), **Transformer** (now universal), MLP (still strong baseline), DLRM (recsys).

Infra: GPU / TPU, mixed precision (bf16 / fp16), distributed (DDP / FSDP), ONNX / TensorRT / Triton for serving; KV-cache for autoregressive.

Tradeoff: beats classical only when data is large + unstructured. On tabular $< \sim 1\text{M}$ rows, GBDT still wins.

Cost reality: training amortized; *inference* is the recurring bill — quantization, distillation, batching matter.

Transfer Learning & PEFT

CHEAP SPECIALIZATION

Problem: reuse a large pretrained model on a small labeled task.

Production: domain BERT (legal, biomed), CLIP for product search, ResNet $\rightarrow$ defect detection, **LoRA / QLoRA** fine-tunes of Llama for vertical assistants.

Patterns: feature extraction (freeze backbone), full fine-tune (rare, expensive), **PEFT** (LoRA, adapters, prefix-tuning — $< 1\%$ params).

Infra: single-node GPU often enough; checkpoint registry; eval harness per domain.

Heuristic: $< 10\text{k}$ labels $\rightarrow$ prompt; $10\text{k} - 1\text{M} \rightarrow$ LoRA; $> 1\text{M} \rightarrow$ full fine-tune or train smaller specialist.

Online Learning

STREAMING UPDATES

Problem: update model continuously as data arrives — non-stationary distributions.

Production: ads CTR (Google, Meta), news ranking, dynamic pricing, real-time anti-abuse.

Models: FTRL-Proximal, Vowpal Wabbit, online GBDT, incremental SGD on shallow nets.

Infra: Kafka / Flink streaming features, warm-state models, shadow eval against batch baseline, kill-switch on degradation.

Risks: feedback loops, poisoning, drift-amplification. Always pair with offline eval + canary.

Active Learning

LABEL EFFICIENCY

Problem: labels are expensive; pick the *most informative* samples to label next.

Production: radiology annotation, legal doc review, content moderation, LLM eval-set curation, autonomous-vehicle edge cases.

Strategies: uncertainty sampling, margin sampling, query-by-committee, diversity sampling, BALD.

Infra: labeling service (Labelbox / Scale / internal) + model-in-the-loop selector + audit trail.

Modern use: curate eval / preference data for LLM fine-tuning & RLHF.

Federated Learning

PRIVACY

Problem: train across data that cannot leave its source (device, hospital, bank).

Production: Gboard next-word, Apple QuickType, hospital-consortium models, cross-bank fraud.

Models: FedAvg, FedProx, secure aggregation, differential privacy.

Infra: on-device training, async aggregation, client sampling, robust to stragglers; privacy budgets tracked.

Reality: niche. Most "privacy" wins come from on-prem training, not true federation.

NLP (classical / pre-LLM)

TEXT UNDERSTANDING

Problem: classification, NER, parsing, summarization, retrieval, QA.

Models: fastText (cheap), BERT / RoBERTa / DistilBERT (encoders), T5 (encoder-decoder), spaCy pipelines.

When still relevant: high-volume classifiers (spam, intent, PII detection) where LLM cost / latency is unacceptable. A distilled BERT at 5 ms beats GPT-4 at 800 ms for routing.

Pattern: LLM generates training data → small model serves traffic.

Computer Vision

PIXELS

Problem: classification, detection, segmentation, OCR, tracking, generation.

Models: ResNet / EfficientNet (classic CNN), YOLO (real-time detection), ViT / Swin (transformer), **SAM** (foundation segmenter), DINOv2 (self-sup features), diffusion (generation).

Infra: GPU inference; for edge use TensorRT, CoreML, OpenVINO; INT8 quantization standard.

Hybrid trend: CNN backbone + ViT head; or SAM / DINO embeddings + small classifier.

Graph ML

RELATIONAL

Problem: nodes + edges (users-products, accounts-transactions, molecules, social).

Production: Pinterest PinSAGE (recsys), Uber Eats restaurant graph, fraud-ring detection at banks & payments, drug-target prediction.

Models: GCN, GAT, GraphSAGE, PinSAGE (sampling-based, scalable), GNNs for KG embeddings.

Infra: distributed graph store (DGL, PyG, Neo4j), sampler workers, GPU training; serving often via cached node embeddings + downstream model.

Trap: graph construction & freshness usually harder than the model.

3. PRODUCTION-CRITICAL PARADIGMS (DETAILED)

3.1 Recommendation Systems

PROBLEM & PRODUCTION

Predict items a user will engage with, given history + context. Drives most consumer-AI revenue: YouTube / TikTok feeds, Netflix homepage, Amazon "you might like", Spotify Discover Weekly, LinkedIn feed, Pinterest.

TWO-STAGE ARCHITECTURE (INDUSTRY STANDARD)

1. **Candidate generation (retrieval)**: reduce 10^8 items $\rightarrow$ 1000. Cheap, recall-oriented. Two-tower NN, ANN over item embeddings, popularity, collaborative filtering.
2. **Ranking**: score 1000 $\rightarrow$ top-N. Expensive, precision-oriented. DLRM / DCN / wide-and-deep, GBDT, transformer rerankers.
3. **Re-ranking / business logic**: diversity, freshness, fairness, ad insertion, contextual bandits for exploration.

MODELS

- **Matrix factorization / ALS** — classic baseline, still strong for cold systems.
- **Two-Tower** — user & item towers $\rightarrow$ embedding similarity $\rightarrow$ ANN. Decouples retrieval cost.
- **Sequential (SASRec, BERT4Rec, GRU4Rec)** — session-aware, captures intent drift.
- **DLRM / Deep & Cross** — handles massive sparse + dense features.
- **Graph (PinSAGE, LightGCN)** — leverages user-item graph structure.

INFRA

- Offline training (Spark + Ray / Horovod) $\rightarrow$ embedding tables to ANN index (Faiss, ScaNN, Vespa, Vald).
- Online: feature store (Feast / Tecton) $\rightarrow$ retrieval (Vespa / Pinecone / Milvus) $\rightarrow$ ranker (Triton, TF-Serving) $\rightarrow$ re-rank.
- p99 budget $\sim$ 100–300 ms end-to-end; ranker dominates.

METRICS

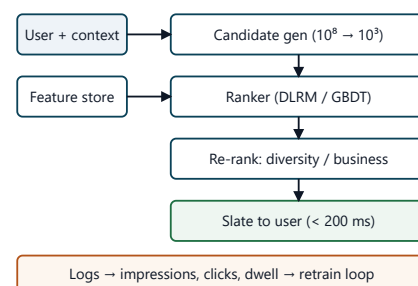
- **Offline**: NDCG@k, MAP, Recall@k, AUC, calibration.
- **Online (truth)**: CTR, conversion, dwell time, session depth — measured via A/B + interleaving.
- **Guardrails**: diversity, novelty, fairness, exposure parity.

FAILURE MODES

- **Feedback loops** — popular gets more popular; mitigate with exploration / bandits.
- **Cold start** — content / user — use content features + heuristics.
- **Position bias** in logged data — debias via IPS or position features.
- **Offline / online mismatch** — wrong features at serve time, train-serve skew.

Interview angle: "Recommendation vs ranking?" — recommendation answers *what to surface from a huge corpus* (recall first), ranking answers *given a candidate set, what order maximizes the objective* (precision first). They compose. **GenAI link**: LLMs help with cold-start (content understanding), re-ranking via reasoning, and natural-language explanations — they rarely replace the two-stage core.

CONSUMER REVENUE ENGINE



Two-stage recsys: retrieve cheaply, rank expensively, re-rank for product rules.

3.2 Ranking Systems (Learning to Rank)

WHERE THE MONEY IS

WHERE IT LIVES

- Web search (Google, Bing) — billions of queries.
- Ads — every impression auction.
- Recsys final stage.
- E-com product search, app stores, marketplaces.

APPROACHES

- **Pointwise** — predict score / CTR per item (regression / classif).
- **Pairwise** — RankNet, LambdaRank, LambdaMART (GBDT). Still SOTA on tabular features.
- **Listwise** — ListNet, SoftRank — optimizes NDCG directly.
- **Neural rerankers** — BERT cross-encoder, ColBERT, ListT5 for top-k re-ranking.

PIPELINE (SEARCH EXAMPLE)



Latency budget grows left $\rightarrow$ right; quality grows right $\rightarrow$ left.

METRICS & TRAPS

- **Metrics**: NDCG@k, MRR, MAP, ERR. Live: CTR, revenue/query, dwell.
- **Traps**: position bias, click bias, selection bias, query intent drift, freshness vs relevance tradeoff.
- **Debiasing**: Inverse-Propensity-Scoring (IPS), counterfactual eval, randomized interleaving.

Interview note: "Why is ranking the heart of most AI products?" — because retrieval is commoditized, but ordering = revenue. **GenAI angle**: LLM cross-encoders give big NDCG lifts on small re-rank sets but are too slow for the L1 ranker — classic L1/L2 split persists.

3.3 Embeddings & Vector Search

FOUNDATION OF MODERN AI

WHY THEY MATTER

Embeddings convert *anything* (text, image, code, user, product, log line) into a dense vector where geometric proximity = semantic similarity. They are the universal interface between unstructured data and ML systems.

PRODUCTION USES

- **Semantic search** over docs, code, support tickets — replaces / augments BM25.
- **Retrieval for RAG** — the "R" in RAG.
- **Recsys candidate generation** — two-tower → ANN.
- **Dedup / clustering** — near-duplicate detection at scale.
- **Multimodal search** — CLIP-style text→image.
- **Anomaly detection** — outliers in embedding space.

MODELS

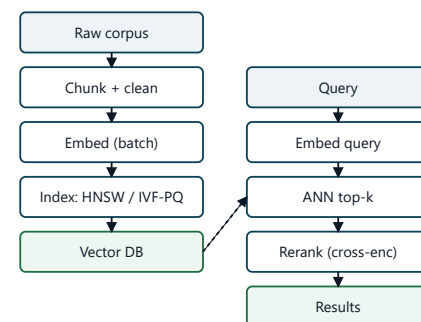
- **Text:** OpenAI text-embedding-3, Cohere embed-v3, BGE, E5, GTE, Voyage. SBERT for cheap on-prem.
- **Multimodal:** CLIP, SigLIP, ImageBind.
- **Code:** Voyage-code, OpenAI code, CodeBERT.
- **Domain finetuned** often outperforms general models by 5–15% NDCG.

INDEX CHOICES

INDEX	BEST FOR
Flat (brute)	<100k vectors, perfect recall
HNSW	Default for <100M, great recall/latency
IVF-PQ	100M–10B, memory-constrained
ScaNN	Google-scale, billion+ vectors
DiskANN	Cold storage, billions on SSD

VECTOR DBS

- **Managed:** Pinecone, Weaviate Cloud, Vertex Matching Engine, Azure AI Search, Turbopuffer.
- **OSS:** Milvus, Qdrant, Weaviate, Vespa (hybrid), LanceDB.
- **Embedded:** Faiss, ScaNN, Chroma (prototyping only).
- **Hybrid (BM25 + vector)** nearly always beats pure vector for enterprise search.



Index pipeline (left) and query path (right). Reranker turns recall into precision.

FAILURE MODES

- **Embedding drift** when re-embedding with new model — must reindex.
- **Chunking** too small (loses context) or too big (poor recall).
- **Domain mismatch** — generic embeddings on technical / legal content.
- **Hot shards** from skewed query distribution.
- **Recall @ k** ignored; teams ship without measuring it.

3.4 Retrieval-Augmented Generation (RAG)

ENTERPRISE GENAI DEFAULT

WHY RAG

LLMs are frozen at training time, hallucinate without grounding, and can't see private data. RAG injects fresh, authoritative context at inference: *retrieve* → *augment prompt* → *generate*.

PRODUCTION PIPELINE

1. **Ingest** — connectors (Drive, Notion, Confluence, Slack, SQL, S3).
2. **Parse + chunk** — semantic chunking, header-aware, overlap; preserve metadata.
3. **Embed + index** — vector DB + BM25 keyword index (hybrid).
4. **Query rewrite** — HyDE, multi-query, decomposition for complex asks.
5. **Retrieve** — hybrid (BM25 + vector + filters) → top-k (typically 20–100).
6. **Rerank** — cross-encoder (Cohere Rerank, BGE-reranker) → top 3–10.
7. **Generate** — LLM with retrieved context + system prompt + citations.
8. **Evaluate** — RAGAS, faithfulness, answer relevance, context precision/recall.

ADVANCED PATTERNS

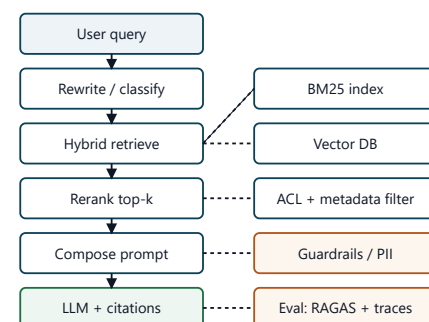
- **Agentic RAG** — LLM plans retrieval (multi-hop, tool use).
- **GraphRAG** — knowledge graph + community summaries for global questions.
- **CRAG / Self-RAG** — model decides when to retrieve, evaluates own answers.
- **Caching** — semantic cache for repeated questions; KV-cache reuse.

METRICS

- **Retrieval:** Recall@k, MRR, Context Precision/Recall.
- **Generation:** Faithfulness, Answer Relevance, Citation Accuracy.
- **Ops:** p95 latency, \$/query, cache hit rate.

WHEN RAG IS THE WRONG TOOL

- Reasoning over the *entire* corpus — use fine-tune or GraphRAG.
- Highly structured data — use text-to-SQL.
- Stable, narrow domain — fine-tune is cheaper at scale.
- Long-tail factual recall — fine-tune embedded knowledge fights RAG.



Hallucination drivers: bad retrieval (top), too-long context, conflicting chunks, missing ACL filter, prompt that doesn't *require* citation, and no faithfulness eval. Most "LLM hallucinates" reports are *retrieval* bugs.

STACK

- Frameworks: LangChain, LlamaIndex, Haystack, Semantic Kernel.
- Vector DB: see §3.3.
- Rerankers: Cohere, BGE, ColBERT.
- Eval: RAGAS, TruLens, DeepEval, Phoenix.
- Observability: LangSmith, Langfuse, Arize, Helicone.

3.5 Time-Series Forecasting

CAPACITY & PLANNING

PRODUCTION USES

- Demand forecasting (Amazon, Walmart, Uber) drives inventory + dispatch.
- Capacity planning (cloud, datacenters) — GCP / AWS internal.
- Financial — pricing, risk, treasury cash flow.
- SRE — predict autoscaling, traffic, alert baselines.
- Energy — load forecasting for grids.

MODEL FAMILIES

- **Statistical:** ARIMA, ETS, Prophet — strong baselines, interpretable, fast.
- **ML / tabular:** GBDT with lag/rolling/calendar features — wins most Kaggle & production benchmarks.
- **Deep:** DeepAR (RNN), N-BEATS / N-HiTS (MLP), **Temporal Fusion Transformer**, PatchTST.
- **Foundation TS:** TimesFM, Chronos, Moirai — zero-shot forecasting; useful for many low-data series.

INFRA & EVAL

- Batch retrain (daily / weekly); online serving usually trivial.
- **Rolling-origin backtest** is mandatory — never random split.
- Hierarchical reconciliation when forecasting at SKU + region + total.

METRICS

- **Point:** MAPE, sMAPE, MAE, RMSE.
- **Probabilistic:** pinball loss, CRPS, coverage of intervals.
- **Business:** stock-out cost, holding cost, capacity over-provision.

Heuristic: Start with Prophet or ETS as a baseline. Add GBDT with lags. Only reach for Transformers when you have many related series and need cross-series learning.

3.6 Anomaly Detection

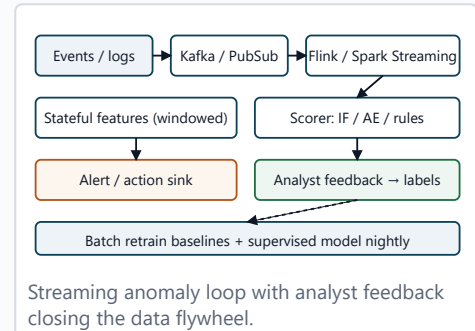
SECURITY + OBSERVABILITY

WHERE IT LIVES

- **Fraud / AML** — Stripe Radar, banks, payment networks (often supervised + unsupervised hybrid).
- **Cybersecurity** — intrusion, lateral movement, EDR (CrowdStrike, SentinelOne).
- **Observability** — Datadog Watchdog, New Relic AI, log anomaly, trace outliers.
- **Industrial** — predictive maintenance on sensor streams.
- **Healthcare** — abnormal vitals, claim anomalies.

MODELS

- **Statistical:** robust z-score, MAD, EWMA, STL decomposition, seasonal-trend residual.
- **Unsupervised:** Isolation Forest (fast tabular default), LOF, One-Class SVM, GMM, DBSCAN.
- **Deep:** Autoencoder reconstruction, VAE, deep SVDD, LSTM-AE for sequences, transformer AE.
- **Supervised** when labels exist — almost always wins. Treat unsupervised as bootstrap.
- **Embedding-based** — distance to nearest neighbor in embedding space.



FAILURE MODES & METRICS

- **Alert fatigue** from low precision — tune for >0.7 precision before recall.
- **Concept drift** — rolling baselines, weekly retrain.
- **Seasonality leaks** as anomalies — decompose first.
- Metrics: Precision@k, PR-AUC, MTTD, MTTR; avoid raw accuracy (data is >99% normal).

3.7 Generative AI

UNSTRUCTURED TASKS

What it solves: producing tokens / pixels / audio that didn't exist — content, summaries, code, conversations, images, speech.

Models: autoregressive LLMs (GPT-4o, Claude, Llama 3, Mistral, Qwen), diffusion (Stable Diffusion, Flux, SDXL), audio (Whisper, Bark, ElevenLabs), video (Sora, Veo).

Infra: vLLM / TGI / TensorRT-LLM for serving; KV-cache, continuous batching, speculative decoding, tensor parallelism. Quantization (AWQ, GPTQ, FP8). For diffusion: latent caching, distilled samplers (LCM, Turbo).

Metrics: BLEU / ROUGE largely obsolete; use **LLM-as-judge**, pairwise human eval, task-specific rubrics, factuality, toxicity, jailbreak rate, \$/token, p95 TTFT & tokens/sec.

What GenAI didn't replace: tabular ML, low-latency classifiers, deterministic ranking, time-series forecasting, fraud scoring, recsys core.

3.8 Multi-Agent AI Systems

EMERGING, BRITTLE

What: multiple LLM-driven agents with tools, memory, and a planner / orchestrator collaborating on a goal.

Patterns: ReAct (think → act), Plan-Execute (planner + worker), Reflexion (self-critique), Supervisor + workers, debate / consensus, hierarchical teams.

Frameworks: LangGraph (state machines), CrewAI, AutoGen, OpenAI Assistants / Swarm, Smol Agents, Mastra.

Production realities: error compounds across steps; latency & cost balloon; debugging via traces (LangSmith, Langfuse, Arize). Treat agents as *workflows with stochastic steps*, not autonomous beings.

Eval: task success rate, tool-call correctness, trajectory rubrics, regression suites; human review still load-bearing.

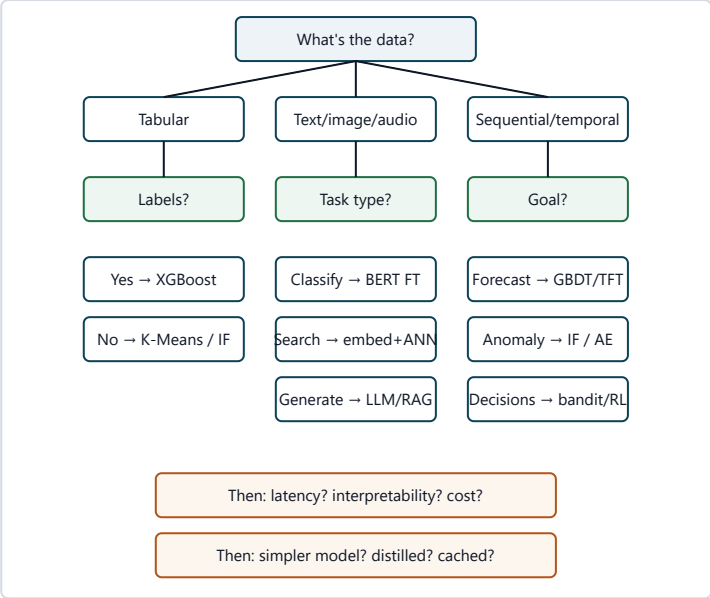
When NOT to reach for agents: single-call prompts solve it; deterministic workflows; latency-sensitive paths.

4. PARADIGM SELECTION FRAMEWORK

"If your problem looks like X → start with Y." Decision heuristics for the first 80% of cases.

PROBLEM SHAPE	START WITH
Labeled tabular data (<10M rows, structured features)	XGBoost / LightGBM
Labeled tabular, very large + sparse (ads, recsys features)	DLRM / DCN / wide-and-deep
Text classification w/ small labeled set	Fine-tune distilled BERT / E5 + linear head
Unstructured text task, no good labels	LLM zero/few-shot → distill to small model
Semantic search over corpus	Hybrid BM25 + embedding + reranker
Enterprise Q&A over private docs	RAG (hybrid retrieve + rerank + LLM + citations)
Personalized feed / catalog	Two-stage recsys (two-tower + DLRM)
Search / ads final ordering	LambdaMART, neural reranker on top
Forecast metric over time	Prophet → GBDT-with-lags → TFT / N-BEATS
Unknown patterns / EDA	K-Means + UMAP, then frame supervised task
Rare event in stream (fraud, intrusion, log)	Isolation Forest / AE → supervised once labels accrue
Sequential decisions with reward signal	Contextual bandit first; RL (PPO) if state matters
Aligning an LLM to preferences	SFT → DPO (cheap) or RLHF (heavier)
Tiny labeled set + large unlabeled	Transfer-learn a foundation model
Relational / graph-shaped data (fraud rings, KGs)	GraphSAGE / PinSAGE
Data can't leave its source	Federated learning + DP
Need explanations / regulator audit	LogReg / GBDT + SHAP, not deep nets
p99 < 10 ms inference	Logistic / GBDT / small NN; never an LLM

Decision tree (quick)



Three guiding questions

- Is there a label or a reward?** No → unsupervised / self-supervised. Reward → RL or bandit. Labels → supervised.
- What's the latency & cost SLO?** Defines whether the answer can be a deep model, an LLM, or must be a distilled small model.
- How fresh must the model be?** Defines batch vs online vs streaming inference, retrain cadence, and feature-store needs.

5. CLASSICAL ML VS DEEP LEARNING VS GENAI

ASPECT	CLASSICAL ML (GBDT / LR / SVM)	DEEP LEARNING (CNN / TRANSFORMER SPECIALIST)	GENAI (FOUNDATION LLM / DIFFUSION)
Data needs	10 ³ –10 ⁷ structured rows	10 ⁵ –10 ⁹ structured / unstructured	10 ¹⁰ + tokens (pretrain); few-shot at inference
Feature engineering	Heavy, domain-led	Mostly learned; some preprocessing	Minimal; prompt & retrieval engineering instead
Interpretability	High (SHAP, coefficients)	Low–medium	Very low (semantic, not mechanistic)
Training cost	\$ (single node, minutes–hours)	\$\$\$ (GPU cluster, hours–days)	\$\$\$\$\$ (1000s GPUs, weeks)
Inference cost	μs–ms, CPU	ms, GPU often	10s–1000s ms; GPU mandatory; \$/call dominates
Latency & throughput	Excellent	Good with optimization	Worst; needs caching, batching, speculative decode
Operational complexity	Low–med	Med–high	High (eval, safety, drift, cost, hallucination)
Retraining frequency	Daily–weekly	Weekly–monthly	Rare full retrain; frequent fine-tune / prompt / index updates
Sweet spot	Tabular, ranking, fraud, forecasting	Vision, speech, large recsys	Language, generation, low-data tasks via prompting
Where it dominates today	~70% of enterprise models	Most consumer perception & recsys	Unstructured text/code/image creation

Practical guidance: tree models dominate tabular because they handle mixed types, missingness, non-linearity, and small data with no tuning. Transformers became universal because attention scales with compute & data better than any prior architecture. Retrieval matters because LLMs are stateless & frozen; embeddings matter because they are the universal interface; observability matters because most production failures are operational, not algorithmic.

6. SYSTEMS & INFRASTRUCTURE

Training vs Inference Pipelines

TWO CLOCKS

Training (offline, hours-days)

Raw data → ETL / dbt → Feature store → Train job → Registry

Inference (online, ms / batch, hours)

Request → Online features → Model server → Post-process → Response

Shared feature definitions enforce train / serve consistency (online-offline parity)

Feature Store

SINGLE SOURCE OF TRUTH

- Why:** kills the #1 ML bug — train/serve skew. One feature definition, two materializations.
- Offline store** (BigQuery / Snowflake / S3 + Parquet) — training, backfills, point-in-time joins.
- Online store** (Redis / DynamoDB / Bigtable / KeyDB) — < 10 ms reads at request time.
- Registry** — feature catalog, owners, freshness SLO, lineage.
- Tools:** Feast (OSS), Tecton, Vertex Feature Store, SageMaker, Hopsworks, Chronon (Airbnb).
- Smell test:** can two teams ship features that share a definition? If not, you don't have a feature store, you have a database.

MLOps Workflow (Data Flywheel)

CLOSED LOOP

Drift detect → Data + labels → Train → Eval / CI → Deploy (canary) → Monitor → Drift detect

Every loop must close: monitoring must produce labels that improve the next train.

- Tooling:** Airflow / Dagster (orchestration), MLflow / W&B (tracking), KServe / Seldon / SageMaker / Vertex (serving), Evidently / WhyLabs / Arize (monitoring).
- Drift:** input drift (PSI, KL), prediction drift, label drift, performance drift. Alert & auto-retrain triggers.
- Shadow + canary + champion/challenger** are the only safe deploy patterns.

Online Inference Stack

LATENCY ENGINEERING

- Model server:** Triton (multi-framework, ensemble), TorchServe, TF-Serving, KServe, BentoML.
- LLM server:** vLLM (paged attn, continuous batching), TGI, TensorRT-LLM, Aphrodite, SGLang.
- Optimization:** quantization (INT8 / FP8 / AWQ / GPTQ), distillation, pruning, KV-cache reuse, speculative decoding.
- Patterns:** autoscaling on GPU utilization & queue depth, replica + tensor parallelism, request routing by model size, semantic cache, prefix cache.
- Observability:** per-model latency / cost / accuracy dashboards; per-tenant quotas.
- Multi-tenant AI platform** requires: model registry, capacity scheduler, isolation, billing per call, eval harness.

Heuristic: If a model serves > 100 QPS and costs > \$10k/mo, it earns 1 week of optimization (quantize, batch, distill). Most "we need more GPUs" is "we never optimized."

PATTERN	WHEN TO USE	TOOLS	KEY TRADEOFF
Batch ML	Daily scoring of large populations (credit, churn, dashboards)	Spark, Ray, Airflow, KubeFlow	Cheap, simple; stale by hours
Real-time inference	Sub-second responses (ads, search, fraud)	Triton, vLLM, KServe	Costlier; complex scaling
Streaming ML	Event-driven features & scoring (IoT, anomaly, recsys updates)	Kafka + Flink / Spark Streaming + Feast	Stateful, hard to test; bursty
Vector / RAG	Semantic retrieval & grounded generation	Pinecone / Milvus / Vespa + vLLM	Indexing cost & embedding drift
Edge inference	Privacy / offline / latency (mobile, IoT, AV)	TensorRT, CoreML, ONNX Runtime, TFLite	Update / monitor harder
Multi-tenant AI platform	Many teams, many models, shared GPU pool	Vertex / SageMaker / KServe + scheduler	Capacity planning, isolation

7. HYBRID AI SYSTEMS (HOW REAL PRODUCTS ARE BUILT)

Almost no real product is a single model. Successful AI systems compose paradigms behind a deterministic backbone.

Enterprise copilot / Q&A

Classical retrieval + embeddings + reranker + LLM + guardrails + analytics.

- Intent classifier (small BERT) → routes to RAG / SQL / tool / chit-chat.
- Hybrid retrieve (BM25 + vector) → cross-encoder rerank → LLM with citations.
- PII filter + jailbreak detector before & after LLM.
- Feedback loop: thumbs / corrections → fine-tune reranker & eval set.

Healthcare / speech-therapy assistant

Rules + ML + vector search + LLM combined to balance reliability and flexibility.

- ASR** (Whisper / domain-tuned) → transcripts.
- Domain classifier flags articulation patterns; embeddings retrieve similar prior sessions.
- Rule engine enforces clinical policy; LLM drafts notes & summaries (always reviewed).
- Audit log + eval harness for safety; PHI never leaves controlled boundary.

Modern observability platform

- Streaming anomaly detection on metrics (Isolation Forest / EWMA per series).
- Log clustering (embedding + DBSCAN) to deduplicate noisy errors.
- LLM summarizes incidents from traces + recent commits + runbooks.
- Forecasting models drive autoscaling & capacity alerts.
- RAG over runbooks & postmortems answers "what should I do?"

E-commerce search & discovery

- BM25 + embedding hybrid retrieval over catalog.
- LambdaMART L1 ranker; BERT cross-encoder rerank top-100.
- Recsys two-tower for "you might also like" injected into slate.
- LLM generates summary cards & comparison; bandits explore new SKUs.
- Forecasting drives stock-aware boosting / suppression.

ML Paradigms — Staff+ Interview Handbook

8 / 9

Cross-cutting orchestration concerns

CONCERN	WHAT TO BUILD	ANTI-PATTERN
Orchestration	Deterministic workflow (LangGraph state machine, Step Functions, Temporal) with LLM nodes as stochastic steps	Free-form agent loops with no exit condition
Retrieval quality	Hybrid retrieval + rerank + filters + ACLs + per-collection eval	"Just embed everything and trust cosine"
Guardrails	Pre-filters (PII, jailbreak), post-filters (toxicity, schema, factuality), tool sandboxes	Trusting the LLM to police itself
Evaluation	Golden set + LLM-judge + human review + production traces sampled	"Looked fine in the demo"
Cost & latency	Routing (cheap model first, escalate), semantic cache, batch where possible, KV reuse	One-size-fits-all flagship model
Safety / compliance	Data lineage, prompt + output logs, region pinning, redaction, opt-out	Sending PHI/PII to third-party APIs without DPA

8. FINAL CHEAT SHEET

A. Which model for which problem

PROBLEM	REACH FOR
Tabular classification / regression	XGBoost / LightGBM
High-cardinality sparse recsys features	DLRM / DCN-v2
Image classification	EfficientNet / ViT (fine-tune)
Object detection	YOLO-v8/9 / DETR
Segmentation (general)	SAM / SAM-2
Text classification (high volume)	Distilled BERT / E5
Text generation / Q&A	LLM + RAG
Semantic search	E5 / BGE + HNSW + rerank
Recsys candidate gen	Two-tower + ANN
Search / recsys ranker	LambdaMART / DLRM
Time-series forecast (general)	GBDT with lags / TFT
Unsupervised anomalies	Isolation Forest / AE
Sequential decisions w/ reward	Contextual bandit / PPO
LLM alignment	SFT → DPO (or RLHF)
Speech to text	Whisper-large-v3 / Distil-Whisper

B. Interview one-liners

- "Most enterprise ML is still tabular and still GBDT."
- "Data quality matters more than model complexity."
- "Retrieval quality often matters more than the LLM."
- "Most ML failures are operational, not algorithmic."
- "Ranking is the hidden core of most AI products."
- "Feature stores exist to kill train/serve skew."
- "Embeddings are the universal interface to unstructured data."
- "LLMs don't replace classifiers — they help build them."
- "Agents are workflows with stochastic steps, not magic."
- "You can't fix what you don't evaluate; build the eval set first."
- "Bandits before RL; PPO before anything fancier."
- "Distill the big model into the small one for prod."
- "Hybrid BM25 + vector beats either alone in enterprise search."
- "If you can solve it with rules, don't use ML."

C. Top production failure modes

- **Drift** — data, concept, label, embedding. Detect early; retrain often.
- **Train/serve skew** — different features or transforms in training vs serving.
- **Stale features** — online store falls behind; freshness SLOs absent.
- **Feedback loops** — model output influences future training data.
- **Skewed training data** — sampling bias, survivorship, geographic, temporal.
- **Poor observability** — no per-model latency, accuracy, or cost dashboards.
- **Latency explosions** — KV cache miss, retry storms, GC pauses, batch starvation.
- **Hallucinations / unsafe outputs** — weak retrieval, missing guardrails, no eval.
- **Cold start** — new user / item / tenant; mitigate with content features & defaults.
- **Cost runaway** — flagship LLMs on cheap traffic; no routing or caching.
- **Eval gaming** — golden set leaks into training; metric goes up, users complain.

D. Questions to ask in ML architecture interviews

DATA & FEATURES

- How do you guarantee train/serve consistency?
- What's your feature freshness SLO?
- Where do labels come from? How clean are they?
- How do you handle PII / PHI in the pipeline?

MODELING & TRAINING

- What's your retraining cadence and what triggers it?
- How do you select between candidate models?
- How do you evaluate retrieval quality independently?
- How do you split data to avoid temporal leakage?

SERVING & OPS

- What's the p99 inference latency and budget breakdown?
- How do you detect & respond to drift?
- What's your canary / rollback strategy?
- How do you monitor cost per request and per tenant?
- For LLMs: how do you measure hallucination & safety in prod?

The one thing to remember: ML maturity is not about which model you use — it's about how fast you can *close the loop* from data → train → eval → deploy → monitor → labels → data. Whoever closes the loop fastest wins.